FULL-DAY WORKSHOP

Successful Semantic Modelling for Power BI

Design, scale, and validate enterprise-grade semantic models, then make them Copilot-ready.

ATTENDEE GUIDE

Phil Seamark & David Browne

Microsoft Fabric Customer Advisory Team

First delivery: DataCon, Seattle · August 2026

Draft v0.1 · 26 June 2026

How to use this guide

This guide is yours to keep. It follows the same order as the day, so you can read ahead, take notes in the margins, and re-run every lab afterwards on your own machine. Each module opens with a divider page, then walks through the concepts, and ends with a hands-on lab you can follow step by step.

Look for these markers throughout the day:

Lab
A hands-on exercise. Follow the numbered steps. Everything you need runs in a browser.

Tip
A shortcut or good-practice nudge worth remembering.

Watch out
A common trap that bites people in real models.

Key message
The one idea to take away from a section.

TIP **YOU ONLY NEED A BROWSER**

We provide the data and the starting models. Claim a numbered user, create your own workspace, copy the `00-setup` notebook into it and run it, then run `0-create-lab-models`. Your environment is then ready, on any operating system.

Your day at a glance

Time (PDT)	Module	You	What you will do
9:00 - 9:20	Welcome, setup, and lab bootstrap	**Hands-on**	Get your own workspace and starting assets
9:20 - 10:05	1. Modelling fundamentals that drive performance	**Hands-on**	Star schema, grain, facts, dimensions, relationships
10:05 - 10:50	2. Storage mode strategy	Watch + exercise	Import vs Direct Lake vs DirectQuery vs Hybrid
10:50 - 11:05	Break		
11:05 - 11:45	3. RLS design done right	**Hands-on**	Dimension-based security, static vs dynamic roles
11:45 - 12:35	4. Scaling techniques	Watch + predict	Aggregations, hybrid tables, partitioning, column splitting
12:35 - 1:30	Lunch		
1:30 - 2:15	5. File health, size, and sorting	Watch	V-Order, sorting, Parquet shape for Import + Direct Lake
2:15 - 3:05	6. Common DAX patterns + the new Calendar feature	**Hands-on**	Period comparison, ranking, ratios, distinct count
3:05 - 3:20	Break		
3:20 - 4:05	7. Diagnosing slow visuals	**Hands-on**	A repeatable triage and tuning workflow
4:05 - 4:45	8. Load testing, proving improvements, and monitoring	**Hands-on**	Prove the gain; Delta Analyzer, BPA, Capacity Metrics
4:45 - 5:00	9. Making your model Copilot-ready	**Hands-on**	Names, descriptions, synonyms, a Copilot answer

Timings are a guide. The **You** column tells you when to have your laptop open. Modules 4 and 5 are watch-only, along with parts of Modules 8 and 9, because they need tooling or a capacity we cannot hand out. In both watch-only modules you will still be asked to *predict* the result before it is revealed, so stay awake. Everything else is yours to do.

Goals for the day

Here is what we want you to walk out with. These are deliberately modest and practical, not a promise to turn you into a performance guru in a day. If you can tick most of these by 5:00, the day was worth your time.

BY THE END OF TODAY YOU SHOULD BE ABLE TO

- Look at a model and name the handful of design choices that make it fast or slow
- Choose a storage mode, and say in one sentence why you chose it
- Add a simple row-level security role and check it with View as role
- Pick one scaling technique that fits a real problem, rather than guessing
- Prove a change made a model faster, instead of just feeling that it did
- Leave with a reusable checklist you can apply to your very next model

TIP **KEEP THIS PAGE OPEN**

We come back to these goals at the end of the day. As each one clicks into place, tick it off. It is a quick way to see the day adding up.

> *Small, concrete goals you can actually use tomorrow beat an ambitious list you forget by Friday.*

MODULE 1

Modelling fundamentals that drive performance

Most performance problems are model problems wearing a DAX costume. Before you tune a single measure, get the shape of the model right.

IN THIS MODULE

Grain · facts vs dimensions · relationships and filter flow · surrogate keys · cardinality and the traps that wreck compression · importing only what you need.

1. Modelling fundamentals that drive performance

BY THE END OF THIS MODULE YOU CAN

- State the grain of a fact table in one sentence, and explain why it matters
- Separate fact responsibilities from dimension responsibilities
- Read how a filter flows from a dimension to a fact across a relationship
- Spot the design choices that quietly destroy compression and query speed

Start with the grain

The **grain** is the meaning of a single row in your fact table: "one row per order line", "one row per daily inventory snapshot", "one row per meter reading". Decide this first and write it down. Every measure, relationship, and filter assumption depends on it. When a model feels confusing, it is almost always because two grains have been mixed into one table.

Facts and dimensions have different jobs

A **fact** table holds the events you measure: it is tall, narrow, numeric, and additive where possible. A **dimension** describes the context you slice by: who, what, where, when. Dimensions are short and wide; facts are long and lean. Keep descriptive text out of fact tables, keep measures out of dimensions, and the model almost designs itself.

Relationships and filter flow

Relationships exist to carry a filter from the "one" side (a dimension) to the "many" side (a fact). Picture the arrows: select a year on the date dimension, the filter flows down to the sales fact, and your measure responds. Single-direction relationships are predictable and fast. Bidirectional relationships feel convenient but make filter flow ambiguous and expensive.

WATCH OUT **BIDIRECTIONAL RELATIONSHIPS**

Reach for bidirectional only when you genuinely need it (for example, a many-to-many bridge), and even then prefer to solve it in the model. A model full of two-way arrows is a model where nobody can predict the answer.

Surrogate keys

Join facts to dimensions on a single, meaningless integer key, not on a natural business key like an email address or a product code. Integer keys compress beautifully, relationships evaluate faster, and you stay insulated from source systems renaming or reusing business codes.

Cardinality is the hidden cost

VertiPaq compresses columns brilliantly, but its job gets harder as the number of distinct values (the cardinality) rises. High-cardinality columns, ultra-wide tables, and deep snowflakes all bloat the model and slow scans. The cheapest performance win is often simply not importing a column you never use.

> **TIP** **IMPORT ONLY WHAT YOU NEED**
>
> Every column you load costs memory and refresh time forever. If a column is not used by a relationship, a measure, or a visual, leave it in the source.

> *Most performance problems are model problems wearing a DAX costume.*

> **LAB 1** **FIND FIVE DESIGN ISSUES**
>
> Approx. 10 minutes · browser · model: `01 Baseline (messy)`
>
> You have been handed a deliberately messy sales model. Before changing anything, your job is to **diagnose**. Open it in the browser and answer the questions, then we will fix them together.
>
> 1. In your workspace, open the `01 Baseline (messy)` semantic model and choose **Open data model** (the web model view).
> 2. Write down the **grain** of the `Sales` table in one sentence.
> 3. Find one table that is acting as **both a fact and a dimension**. What gives it away?
> 4. Identify a **bidirectional relationship**. What ambiguity could it cause?
> 5. Spot a relationship built on a **natural (text) key** instead of a surrogate key.
> 6. Using the field list, find the **highest-cardinality column** you suspect is unused.
>
> **Capture your five findings on the answer sheet overleaf.**

Lab 1: your answers

One or two lines each is plenty. Question 1 is just opening the model, so there are five findings to capture. They are numbered here to match the questions.

2. The **grain** of `Sales`, in one sentence

3. The table acting as **both a fact and a dimension**, and what gives it away

4. The **bidirectional relationship**, and the ambiguity it could cause

5. The relationship built on a **natural (text) key**

6. The **highest-cardinality column** you suspect is unused

LAB 1B **BUILD THE CLEAN STAR**

Approx. 14 minutes · browser (New semantic model) · your own lakehouse

You already have the finished reference in your workspace, `01 Star Schema (fixed)`. Now build your own version to match it. The value is in the doing, wiring the relationships, marking the date table, setting the sort-by, so the moves stick in your hands. This is not a test: if you get stuck at any step, open the reference and follow along, or simply watch. The later labs each come with their own ready-made model, so nothing here can put you behind.

1. Open **your own lakehouse** (the one with shortcuts to the shared data) and click **New semantic model** on the ribbon. Launching it from the *lakehouse* view gives you **Direct Lake over OneLake** (from the SQL endpoint view you would get DL/SQL).
2. Name it `My Star Schema` (so it sits beside the reference), tick the five clean tables, `Date`, `Product`, `Customer`, `Territory`, `Sales`, and Confirm.
3. In **Open data model**, create the five relationships (all Many-to-one from `Sales`): `OrderDateKey → Date[DateKey]` (active), `ShipDateKey → Date[DateKey]` (**make it inactive**), `ProductKey`, `CustomerKey`, `TerritoryKey` to their dimensions.
4. Add `Total Sales = SUM(Sales[SalesAmount])` and mark `Date` as a date table.
5. Set `MonthYear`'s **Sort By Column** to `MonthYearSort` so it sorts chronologically.
6. Prove it works: click **New report** on the ribbon to open a report canvas in the browser, then put `Total Sales` by `Date[MonthYear]` into a column chart. You do *not* need to save the report, it is only there to prove the model behaves.

Done looks like: the chart shows sales by month, the months run in *calendar* order rather than alphabetically (that is your Sort By Column working), and the totals match the `01 Star Schema (fixed)` reference. If the months read "Apr, Aug, Dec...", revisit step 5. If the totals look wrong, check your relationships in step 3.

MODULE 2

Storage mode strategy

Choose Import, Direct Lake, DirectQuery, or Hybrid by workload, with Direct Lake as the default Fabric story.

IN THIS MODULE

Import vs Direct Lake vs DirectQuery vs Composite · Direct Lake first · OneLake vs SQL endpoint · transcoding and column paging · DirectQuery fallback · latency, concurrency, freshness, cost · Hybrid tables · a reusable decision matrix.

2. Storage mode strategy

BY THE END OF THIS MODULE YOU CAN

- Describe how Import, Direct Lake, DirectQuery, and Composite each serve data
- Explain why Direct Lake is the default Fabric choice, and when it is not
- Recognise what triggers DirectQuery fallback and why it is not something to fear
- Choose a storage mode from an SLA, refresh window, and query shape

Four ways to serve the same data

Import loads a compressed copy into memory: fastest queries, but you pay a refresh cost and hold a full copy. **DirectQuery** leaves the data at source and translates every visual into a source query: always current, but only as fast as the source. **Direct Lake** reads Delta/Parquet straight from OneLake into the VertiPaq engine on demand, giving Import-like speed without a refresh step. **Composite** mixes modes in one model, for example a DirectQuery detail table with an Import aggregation over it.

Direct Lake first

On Fabric, start from Direct Lake and justify anything else. It gives you the query performance of Import with the freshness of reading the lake directly, and it removes the refresh window that dominates large Import models. Position the other modes as the exceptions: Import when you need full engine control or you are not on Fabric, DirectQuery when data must never be copied, Composite when one table has a genuinely different profile from the rest.

On OneLake vs on a SQL endpoint

Direct Lake on OneLake reads the Delta tables directly and is the leanest path. Direct Lake on a SQL endpoint goes through the warehouse/SQL analytics endpoint, which adds capability but also a layer. Know which one you are on, because it changes how freshness and permissions behave.

How Direct Lake actually serves a query

The first time a column is needed it is **transcoded** from Parquet into VertiPaq form and paged into memory; after that it behaves like an Import column. This is why a cold query can be slower than the warm queries that follow. Column-on-demand paging means you only pay for the columns you touch, which rewards narrow, well-shaped tables.

DirectQuery fallback is not a failure

If a Direct Lake query cannot be served from memory, for example it exceeds a capacity guardrail, the engine quietly falls back to DirectQuery for that query. The result is still correct; it is just slower. Treat fallback as a signal to check your file layout and guardrails, not as an error to panic about.

What Hybrid tables solve

A Hybrid table keeps most history in Import partitions and the most recent slice in DirectQuery, so you get fast historical queries and near-real-time recent data in one table. It is the answer to "I need years of history and up-to-the-minute today" without importing everything constantly.

> **WATCH OUT** **DIRECT LAKE CAPACITY GUARDRAILS**
>
> Each SKU caps columns, tables, model size on disk, Parquet files per table, and memory. Cross a guardrail and queries fall back to DirectQuery. The guardrails, not your optimism, decide when Direct Lake stays in memory. Keep the reference sheet handy.
>
> **The current numbers, by SKU:**
> learn.microsoft.com/fabric/fundamentals/direct-lake-overview#fabric-capacity-requirements

A decision matrix you can reuse

Workload	Freshness need	Sensible default	Why
Small, high-concurrency finance	Daily	Import (or Direct Lake)	Small enough to import; concurrency loves in-memory
Near-real-time operational	Minutes	Hybrid or DirectQuery on the hot slice	Recent data must be live; history can be fast
Very large historical, hot recent	Hourly/daily	**Import + Hybrid**	Hybrid partitions need an Import base. Direct Lake does not support hybrid tables, so a live hot slice puts you in Import
Very large historical, no live slice	Hourly/daily	Direct Lake	Avoids the refresh window; scales with the lake

EXERCISE 2 **PICK A MODE, DEFEND IT**

Approx. 5 minutes · discussion · no file needed

For each scenario below, choose a storage mode and write one sentence justifying it. We will compare answers and build a shared decision matrix.

1. A 2-million-row finance model, 400 concurrent users, refreshed overnight.
2. An operations dashboard that must show events from the last five minutes.
3. A 3-billion-row sales history where only the current quarter changes.

> *Lead with Direct Lake, size against the guardrails, and let the workload pick the exception.*

MODULE 3

RLS design done right

Secure on a dimension and let the filter flow to the facts. Keep expressions simple. Static vs dynamic roles.

IN THIS MODULE

Secure the dimension, not the fact · avoid `LOOKUPVALUE` on facts · keep the expression trivial · static vs dynamic roles · the mapping-table pattern with `USERPRINCIPALNAME()` · validating with View as role.

3. RLS design done right

BY THE END OF THIS MODULE YOU CAN

- Apply row-level security on a dimension and let it flow to the facts
- Explain why RLS in a semantic model travels through relationships, unlike a database
- Explain why filtering the fact table directly is the slow, fragile path
- Choose between a static role and a dynamic mapping-table role, and build both
- Validate what a user can see with View as role, and know who RLS does not apply to

Secure the dimension, not the fact

Put the security filter on a dimension, for example `Territory` or `Customer`, and let the existing relationship carry it down to the fact. The engine already knows how to flow a filter from the one side to the many side; RLS is just one more filter riding those rails. Filtering the fact table directly throws away that leverage and runs a heavier predicate on every query.

RLS here travels through relationships

This is the biggest difference between security in a semantic model and security in a database, and it is worth pausing on. A relational database has no concept of a relationship the engine can follow at query time, so a security policy has to be defined on *every* table holding sensitive data. Miss one and you have a leak.

A semantic model is not like that. An RLS filter propagates along relationships exactly like any other filter. Secure `Territory` and every fact table related to it is secured too, automatically, because that is simply how filters move. One rule on one dimension can protect the whole model. That is why the advice is to secure the dimension, and it is why RLS here is usually a much smaller piece of work than people coming from a database background expect.

The flip side is that your security is only as good as your relationships. If a fact table is not related to the secured dimension, it is not protected. Orphan tables are a security problem here, not just a modelling one.

Avoid LOOKUPVALUE on the fact table

Reaching for `LOOKUPVALUE` inside an RLS rule on the fact table is a common cause of slow, brittle security. It forces a row-by-row lookup on the largest table in the model, on every query, for every user. If you find yourself writing it, step back: there is almost always a dimension you can secure instead.

Keep the expression trivial, and know why

An RLS filter is not evaluated once. It gets attached to **every storage engine query that touches any table carrying the filter**, for every user, for the life of the model. That is the reason simplicity matters here more than anywhere else in the model.

Done well it can make queries *faster* than having no RLS at all, which surprises people. A simple equality filter like `[Region] = "NZ"` cuts the rows the engine has to scan, and the engine can usually resolve it with a bitmap index, which is close to free. You are handing the storage engine a cheap way to do less work.

Done badly it goes the other way, hard. Nested conditions, string manipulation and multi-table gymnastics cannot take that cheap path, so the engine evaluates the expression the expensive way on every scan, on every query, for every user. That is how a security rule quietly maxes out a capacity. Complexity in the security layer is the most expensive complexity you can add.

The two kinds of RLS

There are two patterns, and picking the right one is most of the design decision.

1. Static RLS. You create several roles and write the filter rule directly on each one. A `USA` role filters `[Country] = "USA"`, a `Canada` role filters `[Country] = "Canada"`, and so on. You then add people to roles individually or, far better, by security group. Someone can belong to more than one role, in which case they see the *union*: put a person in both roles above and they see USA and Canada sales together. The rules live on the role, so the model itself carries the definition of who sees what.

It is simple, obvious to read, and perfectly good for a handful of slices. It stops being practical once you have dozens of territories, because every new slice means a new role and a model change.

2. Dynamic RLS. You keep *one* role, and move the decision into the data. A control table holds rows describing who can see what: one row saying Phil / USA, another saying David / Canada. The role filter is just `[UserEmail] = USERPRINCIPALNAME()`, which matches the signed-in user to their rows, and the relationship carries that through to the facts.

The model no longer knows or cares who sees what, the table does. Granting someone access is an `INSERT`, not a model change and republish, which is what makes this the pattern that scales.

TIP **THE DYNAMIC MAPPING PATTERN**

Create a `UserAccess` table (user email + the dimension key they may see). Relate it to the dimension. The role filter becomes simply `[UserEmail] = USERPRINCIPALNAME()` . To change access, edit a row, not the model.

WATCH OUT **RLS DOES NOT APPLY TO ADMINS**

RLS only bites for people with **read** access. Workspace Admins, Members and Contributors, and the owner of the model, bypass it entirely and see everything. This catches people out constantly: you finish building RLS, open the report yourself, see all the data and conclude it is broken. It is not, you are just an admin.

The way to check is to test as someone else. In Power BI Desktop that is **View as**, where you can pick a role and also enter **Other user**. In the service, open the semantic model's **Security** page and use **Test as role**, which also lets you name a specific user. Testing as a user rather than a role is the only way to prove a *dynamic* rule really works, because it exercises the control table lookup rather than just the role definition.

TIP **DEFINE IT ONCE, IN ONELAKE**

If your model is **Direct Lake on OneLake**, rules defined with **OneLake security** propagate through to the semantic model, so you do not have to define them a second time in the model at all. The same rules also apply to everything else reading that data, Spark, the SQL endpoint, other models, which is the real prize: one definition enforced everywhere, rather than one per engine and the drift that comes with it.

Note this is specific to Direct Lake *on OneLake*. Direct Lake on a SQL endpoint and Import both still need the rules defined in the model as normal, which is one more practical consequence of the Module 2 distinction. Check where the feature has got to on your tenant before you plan around it.

WATCH OUT **BIDIRECTIONAL FILTERS AND RLS**

RLS plus bidirectional relationships is where surprising results and slow queries breed. Keep relationships single-direction where you can, and test every role. The one deliberate exception is the mapping table itself: it sits on the *many* side, so its relationship to the dimension must be **bidirectional** with **Apply security filter in both directions** on, or the role filters the mapping table and stops there, and everyone quietly sees everything. The engine will not accept the security setting on a single-direction relationship, so the two travel together.

> *Secure on the dimension, keep the expression trivial, and let relationships do the work.*

LAB 3 **ADD A STATIC AND A DYNAMIC ROLE**

Approx. 10 minutes · browser · model: `03 RLS`

1. Open the `03 RLS` model and choose **Open data model**. Under **Manage roles**, create a static role `NZ Only` on the `Territory` table: `[Region] = "NZ"` .
2. Use **View as role > NZ Only** and confirm the sales visuals now show only NZ.
3. Create a second role `Dynamic Territory` on the `UserAccess` table: `[UserEmail] = USERPRINCIPALNAME()` .
4. Use **View as role** with a mapped user and confirm the filter flows to the facts. If you see *every* region, the security filter on the `UserAccess` relationship is not flowing both ways.
5. Note the difference: to change who sees what, you edited a *row*, not the model.

MODULE 4

Scaling techniques

Aggregations, hybrid tables, split columns, and partitioning, framed as responses to specific pain.

IN THIS MODULE

Match the pattern to the pain · user-defined aggregations · hybrid tables for hot/cold · incremental refresh and partitions · splitting date and time columns · Direct Lake equivalents: V-Order, file layout, guardrails.

4. Scaling techniques

BY THE END OF THIS MODULE YOU CAN

- Pick the right scaling pattern from the symptom you are seeing
- Add a user-defined aggregation and confirm queries hit it
- Use hybrid tables and partitioning to cut refresh and query cost
- Shape columns so they compress instead of bloat

Match the pattern to the pain

Scaling is not one trick; it is a small toolbox where each tool answers a specific symptom.

Symptom	Reach for
Queries scan far more detail than the visual needs	User-defined aggregations
Refresh window is too long	Incremental refresh and partitioning
Need history fast and recent live	Hybrid tables
A single wide column wrecks compression	Split columns / reshape

User-defined aggregations

An aggregation table pre-summarises the fact at a coarser grain, for example daily totals per product. When a visual asks for something the aggregation can answer, the engine silently serves it from the small table instead of scanning the huge one. Detail stays available for drill-through; the common questions just get cheaper.

Hybrid tables for hot and cold

Store cold history in Import partitions and the hot recent slice in DirectQuery. You keep fast queries over years of data while today stays near-real-time, all inside one table the report author never has to think about.

The part that makes it pay off is the **data coverage definition**, an expression on the DirectQuery partition that tells the engine which rows that partition holds. With it in place, a query that cannot possibly need the DirectQuery half skips it entirely and runs at Import speed. One catch worth remembering: a range expression like `< 20260101` has to sit on a **dual** table, so you point it at the date dimension rather than the fact. The engine needs an in-memory copy it can check cheaply before deciding whether to reach out to the source.

Incremental refresh and partitions

Partition the fact by date and only refresh the partitions that changed. A model that used to reload everything nightly can instead refresh just the last few days, turning an hour into minutes and taking load off the source.

Split columns to help compression

A single high-cardinality `datetime` column can be the most expensive column in the model. Split it into a `date` and a `time` column and each compresses far better, because each has far fewer distinct values. The same idea applies to composite keys and concatenated text.

The Direct Lake equivalents

In Direct Lake you do not refresh, so scaling shifts to the file layer: V-Order and sorting on load, sensible Parquet file and rowgroup sizes, and staying inside the capacity guardrails. Module 5 goes deep on this; the mindset is the same, the levers just move upstream into the lake.

> **TIP** **CHANGE ONE LEVER, THEN MEASURE**
>
> Each of these can help or hide a problem. Add one, re-measure with the Lab 7 notebook, and keep it only if the numbers moved. Stacking three changes blind is how models get complicated without getting faster.

> *Every scaling technique is an answer to a specific pain. Name the pain first.*

WATCH **FOUR LEVERS, FOUR PAINS**

Approx. 45 minutes · laptops can stay shut · presenter demo

This module is a demo. Configuring a composite model with aggregations needs tooling we are not asking you to install, so your presenter drives and you predict. **Before each reveal, write down the improvement you expect.** Then see how close you were.

1. **Aggregation, hit.** A query grouped by date and product answered from a table about seven times smaller than the detail fact. Watch the storage-engine time, not the total.
2. **Aggregation, miss.** The same query grouped by *customer*. The aggregation has no customer column, so it cannot help and the query falls through to the detail fact. This is the half people forget.
3. **Hybrid and partitioning.** The same filter, written two ways. One prunes the DirectQuery partition, one does not, and the difference is entirely in how the filter was written.
4. **Column splitting.** One high-cardinality decimal column against the same values split in two. Same answer, a fraction of the memory.

Ask for the numbers if you want them written down. Everything shown here is reproducible on the models you already built.

MODULE 5

File health, size, and sorting

Get Delta and Parquet files in shape: V-Order, sorting, and rowgroup sizing for Import and Direct Lake.

IN THIS MODULE

Why file layout matters in both modes · what Import resolves for you · why Direct Lake needs deliberate care · V-Order · sorting on load · rowgroup and Parquet file sizing · OPTIMIZE and V-Order maintenance.

5. File health, size, and sorting

BY THE END OF THIS MODULE YOU CAN

- Explain why file layout affects both Import and Direct Lake, differently
- Describe what V-Order does and why sorting on load matters
- Recognise Parquet files and rowgroups that are too small or too large
- Run OPTIMIZE and V-Order maintenance to keep a table healthy

Why file layout matters

VertiPaq loves data that arrives sorted and well laid out, because similar values sit together and compress harder. In Import you pay this cost once at refresh. In Direct Lake the file layout *is* the model: the engine reads the Parquet you landed, so a messy table is slow on every single query.

Import forgives, Direct Lake remembers

Import re-encodes and re-sorts the data as it loads, so it can quietly rescue a poorly organised source. Direct Lake does not get that second chance. Whatever shape the Delta table is in when you land it is the shape every query has to live with. That is the single biggest mindset shift when you move to Direct Lake.

V-Order

V-Order is a Microsoft optimisation that rewrites Parquet so the VertiPaq engine can read and compress it far more efficiently. It is the difference between a Direct Lake table that flies and one that limps. Make sure V-Order is on for the tables your semantic model reads.

> *Seamark's Maxim: V-Order costs you once on every write and repays you on every read. Apply it where reads outnumber writes.*

Which is not everywhere. V-Order takes longer and costs more compute to write, so in a layered ETL it is worth asking what the read/write ratio actually is at each hop:

Layer	Written	Read by	V-Order?
Bronze / Load	Once	Silver, once	No
Silver / Stage	Once	Gold, once	Rarely
Gold / Persist	Once	Every report render and every DAX query, by every user, all day	**Always**

Bronze and Silver are typically written once and read once by the next stage, so the extra write cost is never earned back. Gold is read thousands of times, so it always is. If you take one setting home from this module, it is this one.

A companion to **Roche's Maxim** from Matthew Roche, "data should be transformed as far upstream as possible, and as far downstream as necessary". That one is about where to transform; this one is about where to V-Order.

Sort on the way in

Sort by the columns you most often filter and group by, typically date and a key dimension, before you write the Delta table. Sorted data clusters similar values, which shrinks the files and lets the engine skip whole rowgroups it does not need.

Rowgroup and file sizing

Too many tiny Parquet files means overhead per file and, in Direct Lake, pressure against the files-per-table guardrail. Files or rowgroups that are too large hurt parallelism and pruning. Aim for healthy, evenly sized files; let compaction consolidate the small ones.

Maintenance is not optional

Delta tables drift as they are appended to. Run `OPTIMIZE` with V-Order on a schedule to compact small files and keep the layout clean, the same way you would rebuild an index. A table that was healthy last month may need a tidy this month.

WATCH OUT **THE FILES-PER-TABLE GUARDRAIL**

Direct Lake caps Parquet files per table by SKU. A trickle of tiny append files can push you over it and force DirectQuery fallback. Compaction is how you stay under the line.

> *In Direct Lake the file layout is the model. Sort and V-Order on the way in, or pay for it on every query.*

PREDICT, THEN WATCH **BEFORE AND AFTER A TIDY**

Your presenter will show a poorly laid-out Delta table, then the same table after sorting, V-Order, and OPTIMIZE. Before they run it, write down three predictions: will the **file count** go up or down, will the **size on disk** go up or down, and will the **query time** go up or down, and by roughly how much. Then watch all three move and check your calls. You do not need to type; the point is to commit to a prediction first.

MODULE 6

Common DAX patterns + the new Calendar feature

Model for period comparison, ranking, ratios, and distinct count, and define a calendar to cut storage-engine scans.

IN THIS MODULE

Period comparisons and row-based time intelligence · the new Calendar feature · ranking · child-to-parent ratios · distinct count as a modelling decision · calculation groups and field parameters.

6. Common DAX patterns + the new Calendar feature

BY THE END OF THIS MODULE YOU CAN

- Set up a date table so period comparisons are simple and fast
- Use the new Calendar feature to cut storage-engine scans without rewriting DAX
- Write ranking and share-of-parent measures that fold well
- Treat distinct count as a modelling decision, not just a formula

Period comparisons start in the model

Prior year, prior month, and moving averages all get easy when you have a proper date table and clean relationships. Where you need more than one date context, for example order date and ship date, use role-playing dates with inactive relationships and activate them per measure rather than duplicating the fact. Row-based time intelligence, where the comparison is precomputed as a column, can turn an expensive filter into a simple lookup.

The new Calendar feature

The Calendar feature lets you mark which columns represent year, quarter, month, and day, and supports both financial and Gregorian calendars. Once the engine understands your calendar, common time calculations run with **fewer storage-engine scans**, and you get the speed-up without rewriting your measures. It coexists happily with calculation groups.

Ranking

Ranking measures (top N products, league tables) are easy to write and easy to write badly. Keep the ranked set as small as the question allows, and rank over a dimension column rather than a fact, so the engine has less to sort.

Child-to-parent ratios

"This product as a share of its category" is the classic ratio pattern. Compute the parent total with the right filter removed (typically with `ALLEXCEPT` or a targeted `REMOVEFILTERS`) and divide with `DIVIDE` so a zero denominator is handled cleanly.

Distinct count is a modelling decision

Distinct count is one of the more expensive operations because the engine cannot simply add values up. If a distinct count is central to your report, design for it: a well-shaped key column and, where it fits, a precomputed aggregation will do more for you than any clever measure.

> **TIP** **CALCULATION GROUPS AND FIELD PARAMETERS**
>
> Use calculation groups to apply one set of time calculations across many measures without copy-paste, and field parameters to let users swap measures or dimensions in a visual. Both cut the number of near-identical measures you maintain.

> *Fast DAX starts with predictable filter propagation. Shape the model, then the measure gets simple.*

> **LAB 6** **MODEL FIRST, THEN MEASURE**
>
> Approx. 14 minutes · browser (web modeling) · model: `06 DAX + Calendar`
>
> 1. Open the `06 DAX + Calendar` model and choose **Open data model**. Confirm the date table is marked as a date table.
> 2. Turn on the **Calendar feature** using the Calendar UX (or the **TMDL view** if you prefer) and mark the year / quarter / month / day columns.
> 3. Add a **Sales vs Prior Year** measure. Use the Lab 7 notebook to check the scan count before and after the Calendar feature is applied.
> 4. Add a **calculation group** for time intelligence (Current, Prior Year, YoY %), using the calc-group UX or TMDL.
> 5. Add a **share-of-category** ratio (`DIVIDE` + `ALLEXCEPT`) and a **Top 5 products** ranking. Note how model shape kept the DAX short.

MODULE 7

Diagnosing slow visuals

A repeatable triage workflow: is it the visual, the DAX, the model, or the source?

IN THIS MODULE

A repeatable triage workflow · a trace notebook (storage vs formula engine) · front-end vs storage-engine bound · instructor tools: DAX Studio, Delta Analyzer, BPA · finding the cheapest fix that actually matters.

7. Diagnosing slow visuals

BY THE END OF THIS MODULE YOU CAN

- Follow a repeatable order when a report is slow, instead of guessing
- Read a Server Timings capture to see if a query is FE-bound or SE-bound
- Use the right tool for each layer: visual, DAX, model, source
- Choose the cheapest fix that moves the number that matters

Triage in a fixed order

When something is slow, resist the urge to rewrite the first measure you see. Work top down:

1. **The visual.** Is it asking for too much, a giant table, dozens of measures, a huge matrix?
2. **The DAX.** Capture the query. Is one measure doing the damage?
3. **The model.** Bad relationships, high cardinality, or a missing aggregation?
4. **The source.** For DirectQuery / Direct Lake, is the slowness downstream in the lake or database?

Read the split: front end vs storage engine

Server Timings breaks a query into **storage engine** (SE, scanning and aggregating data) and **formula engine** (FE, the single-threaded step that stitches results together). SE-bound usually means you are scanning too much: fix the model, add an aggregation, cut cardinality. FE-bound usually means the measure logic is expensive: simplify the DAX. The split tells you which half of the problem to work on.

Your tool today: a trace notebook

Everyone can diagnose in the browser: a **Fabric notebook** runs a query, traces it, and prints the **SE/FE split**, the same information Server Timings shows, with nothing to install. Both analyzers run in a notebook too, as `labs.vertipaq_analyzer()` and `labs.delta_analyzer()`, each rendering a proper interactive report. Your presenter will also demo the desktop tools, **DAX Studio** for a richer split and **BPA** for design smells. You do not need them today, but it is worth knowing they exist.

TIP **WHICH ANALYZER, AND WHEN**

VertiPaq Analyzer describes the model *in memory*: structure, cardinality, relationships, and on Import the column sizes too. **Delta Analyzer** describes the data *on disk*: Parquet files, rowgroups and how well it is laid out. On **Direct Lake** the two split apart, because memory only holds the columns that have been paged in, and the size columns are not reported. So on Direct Lake reach for **Delta Analyzer** for anything about size or footprint, and VertiPaq Analyzer for everything about shape. Every model today is Direct Lake, so both get used, for different questions.

WATCH OUT **TUNING THE WRONG LAYER**

Rewriting DAX on an SE-bound query, or reshaping the model on an FE-bound one, burns time and moves nothing. Read the split first, then act.

> *Diagnose in order, read the SE/FE split, then spend your effort where the time actually is.*

LAB 7 **CLASSIFY THREE SLOW QUERIES**

Approx. 15 minutes · browser (notebook) · model: `07 Slow Visual Triage`

1. Open the `lab07-diagnose-slow-visuals` notebook and point it at your `07 Slow Visual Triage` model.
2. Run the three query cells. The notebook traces each and prints the **SE/FE split**.
3. Classify each as **SE-bound** or **FE-bound**.
4. Match each to its root cause: a bad model, a bad measure, or a cardinality/compression problem.
5. Apply the cheapest fix for one in the web model, then re-run its cell. Did the split shift as you expected?

MODULE 8

Load testing, proving improvements, and monitoring

Baseline, change one thing, re-measure, and watch the model in production.

IN THIS MODULE

Anecdote is not proof · benchmark discipline · load testing on Fabric vs locally · concurrency and repeatability · monitoring with Delta Analyzer, BPA, Log Analytics, and Capacity Metrics · a lightweight operating cadence.

8. Load testing, proving improvements, and monitoring

BY THE END OF THIS MODULE YOU CAN

- Run a fair benchmark: baseline, one change, re-measure, record
- Benchmark a query fairly in a browser notebook, cache cleared
- Read the signals that tell you a change actually helped
- Set up basic ongoing monitoring for a production model

Anecdote is not proof

"It feels faster" is where tuning goes to die. Warm caches, a quiet capacity, and wishful thinking all make things feel better than they are. If you cannot show a before and an after number, you have not proven anything.

Benchmark discipline

Four steps, every time: **baseline** the current query, **change one thing**, **re-measure** under the same conditions, and **record** the result. Clear the cache between runs so you compare like with like, and change only one lever at a time so you know what caused the movement.

Load testing: the presenter demo vs your notebook

On a capacity, a Fabric notebook can fire many queries in parallel to simulate real concurrency; your presenter will demo this. For your own hands-on you use a simpler **browser notebook** that clears the cache and measures one query before and after a change. The discipline is identical, only the scale differs.

Concurrency and repeatability

A query that is quick for one user can fall over under a hundred. Test at a realistic concurrency, and run each test more than once, because a single run can be lucky or unlucky. Repeatable numbers are the ones you can defend.

Monitoring once it is live

Delta Analyzer (or **VertiPaq Analyzer** on an Import model) and **BPA** keep the model design honest. **Log Analytics** and the **Capacity Metrics** app show how it behaves in production: which queries are slow, when the capacity is under pressure, and whether things are drifting. Monitoring turns a one-off tune-up into an ongoing habit.

TIP **A LIGHTWEIGHT CADENCE**

You do not need a heavy process. A monthly glance at Capacity Metrics, a BPA pass after any significant model change, and a saved baseline query per key report will catch most regressions before your users do.

> *If you cannot prove the improvement, you do not yet own the improvement.*

LAB 8 **PROVE ONE IMPROVEMENT**

Approx. 10 minutes · browser (notebook) · model: `04 Scaling` or `07 Slow Visual Triage`

1. Open the `lab08-prove-the-improvement` notebook and point it at your model.
2. Run the **baseline** cell; it clears the cache first so the number is fair.
3. Apply a **single** change (an aggregation, a simpler measure, or a model tweak) in the web model.
4. Run the **after** cell; the notebook computes the total change and improvement %.
5. Record both numbers and the SE/FE split in the metrics sheet, then state your one-sentence proof.

MODULE 9

Making your model Copilot-ready

Friendly names, descriptions, and synonyms so both humans and Copilot get good answers.

IN THIS MODULE

Friendly names · descriptions · synonyms · hiding the junk · logical grouping · verified answers and the linguistic schema · why good modelling helps humans and AI alike.

9. Making your model Copilot-ready

BY THE END OF THIS MODULE YOU CAN

- Name and describe objects so a person or Copilot can understand them
- Add synonyms so real-world wording maps to your fields
- Hide the technical clutter that confuses both humans and AI
- Run your model through a final "is it ready?" checklist

Names, descriptions, synonyms

Copilot reads your model the way a new colleague would. `Sales Amount` beats `fct_sls_amt`. A one-line **description** on a measure tells the AI what it means and when to use it. **Synonyms** map the words your business actually uses ("revenue", "turnover", "takings") onto the field, so a natural question finds the right answer.

Hide the junk

Surrogate keys, helper columns, and staging fields should be hidden. Every object you leave visible is one more thing a person, or Copilot, can pick by mistake. A clean field list is a form of documentation.

Group things logically

Organise measures into display folders and keep related fields together. Semantic grouping helps humans browse and gives Copilot useful structure to reason over. The tidier the model, the better the answers.

Verified answers and the linguistic schema

For the questions that really matter, you can define **verified answers** so Copilot responds with a known-good result, and tune the **linguistic schema** so phrasing maps cleanly to your model. This is how you move from "usually right" to "reliably right" on your key questions.

> TIP **GOOD MODELLING WAS THE WORK**
>
> Most of what makes a model Copilot-ready is the same work that makes it good for people: clear names, a clean star schema, sensible measures. If you did the earlier modules, you are most of the way there already.

> *A model that answers well for people answers well for Copilot. The metadata is the last mile.*

LAB 9 **POLISH THE METADATA**

Approx. 8 minutes · browser (web model view) · your golden-state model

1. Rename two cryptic fields to friendly, business-friendly names.
2. Add a one-line **description** to your three most-used measures.
3. Add **synonyms** for one measure and one dimension.
4. **Hide** every surrogate key and helper column.
5. Run the end-of-day checklist: is it fast, explainable, monitorable, and Copilot-friendly?

How did we do?

Back to where we started. This is the same list of goals from this morning. Run down it and be honest: these were meant to be achievable in a day, so most should now feel within reach. Anything still fuzzy is a great place to point your practice next week.

This morning's goal	Can you do it now?
Name the design choices that make a model fast or slow	
Choose a storage mode and justify it in one sentence	
Add a row-level security role and check it with View as role	
Pick a scaling technique that fits a real problem	
Prove a change made a model faster	
Leave with a reusable checklist for your next model	

TIP **ONE THING TO TRY ON MONDAY**

Pick a single model you own and apply just one idea from today: state its grain, add an aggregation, or measure a query with Server Timings. One applied idea beats a notebook full of intentions.

> *The day was worth it if you leave able to do one thing you could not do this morning.*

Your takeaway checklist

Use this to sanity-check any model you build after today.

Question	Yes / No
Can I state the grain of every fact table in one sentence?	
Is it a clean star schema, with no table doing two jobs?	
Are relationships single-direction on integer surrogate keys?	
Did I choose the storage mode deliberately, for this workload?	
Is RLS on a dimension, with a simple expression?	
Are the underlying files in good shape (sorted, V-Ordered)?	
Can I prove the model is fast, with a before/after measurement?	
Is the model explainable to a person and to Copilot?	

> *Take a model from "works" to "scales", and prove why it scales.*